

신입 개발자 · 서비스 개발 → 네트워크·보안 · [GitHub 프로필](#) · [공개 소개 사이트](#)

프로젝트마다 맡은 역할과 담당 범위, 상황에서 판단과 행동을 거쳐 결과에 이르는 과정을 확인 가능한 사실로 정리했습니다.

13 번 과제 · ExplainSOC 보안 AI 성능 숫자 검토 도구

2026.09

역할 기획 · 프론트엔드 구현 · 사용자 관찰 · 검증 · 배포

담당 판독 규칙 15 개·결론 카드·논문 실험실 구현, 사용자 반응 기록 6 회, 재현 ZIP 검증, GitHub Pages 배포
능력 자기조절력

상황 보안 AI 제품 소개서는 '정확도 99%' 같은 숫자를 앞세우지만 그 숫자가 어떤 시험에서 나왔는지는 잘 적지 않습니다. 10 번 논문에서도 같은 XGBoost 가 무작위로 나눈 시험에서는 Macro F1 0.9979, 학습에 없던 공격만 둔 시험에서는 0.3871 이었습니다.

판단 숫자의 좋고 나쁨을 판정하는 점수·등급·합격선은 만들지 않고, 숫자를 만든 시험 조건을 먼저 확인한 뒤 빠진 근거를 판매 업체에 물을 질문으로 바꾸게 하기로 했습니다.

행동 소개서 문장과 성능표에서 빠진 시험 조건을 판독 규칙 15 개로 찾아 공급자 질문으로 바꾸고, 결론 카드와 논문 결과를 직접 바꿔 보는 논문 실험실을 만들었습니다. 기능을 늘리기보다 처음 보는 사람의 반응을 여섯 번 기록하며 막힌 곳을 하나씩 고쳤고, 서버 없이 브라우저에서만 동작하게 했습니다.

결과 ALEPH 동기가 휴대폰에서 도움 없이 업체 질문까지 갔고, 단위·컴포넌트 테스트 152 개와 실제 브라우저 테스트 35 개를 통과한 커밋만 배포되게 했습니다.

기술 React 19 · TypeScript · Vite 8 · Vitest · Playwright

[앱 체험](#) · [공개 저장소](#)

CLOV 팀 프로젝트

2026.06 - 2026.08

역할 팀장 · 풀스택 개발자 · 4 인 팀

담당 서비스 구조 설계, React 화면과 Spring Boot API 구현, Ubuntu·Docker·Nginx·HTTPS 배포, 이슈와 코드 리뷰 운영

능력 대인관계력

상황 비전공자 세 명이 포함된 팀의 첫 모임에 두 명만 참석했고, 서로 다른 AI 도구에서 만든 코드가 자주 충돌했습니다.

판단 설명보다 먼저 기능을 볼 수 있는 프로토타입을 만들고, 작업 전 이슈와 머지 전 실행 확인·코드 리뷰를 팀의 공통 기준으로 정했습니다.

행동 기능을 볼 수 있는 프로토타입을 먼저 만들고 이슈 작성, 실행 확인, 코드 리뷰 뒤 머지하는 흐름을 운영했습니다.

결과 2026 년 8 월 4 일까지 이슈 152 개와 PR 266 개를 기록했고, 직접 리뷰한 PR 72 개를 포함해 서비스를 HTTPS 환경에 배포했습니다.

기술 React · Spring Boot · Docker · Nginx · OAuth

[프론트엔드 저장소](#) · [백엔드 저장소](#)

10 번 과제 · 네트워크 침입 탐지 연구

2026.09

역할 연구 설계 · 구현 · 검증 · 논문 작성

담당 연구 설계, 데이터 감사, 세 모델 학습과 비교 실험, SHAP 설명 안정성 검증, 논문과 재현 자료 공개
능력 자기동기력

상황 무작위 분할에서 높은 탐지 점수가 실제로 처음 보는 공격에 일반화되는지 확인할 필요가 있었습니다.

판단 무작위 분할과 날짜 기반 미관측 공격 분할을 따로 설계하고, 세 모델의 성능과 SHAP 설명 안정성을 같은 조건에서 비교했습니다.

행동 CICIDS2017 2,830,743 건을 감사하고 무작위 분할과 미관측 공격 분할에서 세 모델을 비교했으며 SHAP 설명 안정성을 확인했습니다.

결과 XGBoost Macro F1 이 0.9979 에서 0.3871 로 낮아지는 격차를 재현 자료와 논문으로 제출했습니다.

기술 Python · scikit-learn · XGBoost · SHAP

[논문 읽기](#) · [공개 저장소](#)

8 번 과제 · 패스키로 잠금 비공개 영역

2026.09

역할 WebAuthn 인증 설계 · 구현 · 검증 · 배포

담당 WebAuthn 등록·로그인 흐름 설계와 구현, 세션과 Origin-CSRF 검증, 데이터베이스 설계, 공개 배포

능력 자기조절력

상황 공개 소개 페이지는 그대로 열어 두면서 비공개 자료는 비밀번호 없이 본인만 볼 수 있게 나눠야 했습니다.

판단 비밀번호를 없애는 대신 기기 패스키로 본인을 확인하고, 공개 화면과 비공개 자료의 경계를 요청 단계마다 검증하기로 했습니다.

행동 WebAuthn 등록과 로그인을 구현하고, 일회용 질문을 데이터베이스에서 한 번만 사용하게 했습니다. 서버 세션, Origin 과 CSRF 검사, 마지막 패스키 삭제 방지까지 요청 단계에서 검증했습니다.

결과 판정 기준 47 개를 모두 충족하고 자동 테스트 50 개를 통과했습니다. 공개 화면의 비밀번호 입력 칸은 0 개이며, 로그인하지 않은 비공개 요청은 401 로 거절합니다.

기술 Java 25 · Spring Boot · WebAuthn4J · PostgreSQL · Vercel

[공개 저장소](#)

7 번 과제 · 플랜두씨 다이어리 계정 인증

2026.09

역할 인증 설계 · 풀스택 구현 · 검증 · 배포

담당 Argon2id 비밀번호 저장, 서버 세션에 묶은 JWT 와 Refresh 토큰 회전, 계정별 데이터 격리, 기존 기록 이전, Docker-Render 배포

능력 자기조절력

상황 공개형으로 시작한 Plan-Do-See 다이어리를 개인 계정 기반으로 바꿔야 했습니다. 서명만 확인하는 JWT 는 로그아웃한 뒤에도 만료 전까지 유효하고, 로그인 화면으로 가리기만 하면 다른 계정의 기록을 읽는 길도 남습니다.

판단 Access JWT 를 서버 세션 행에 묶어 매 요청에서 세션 생존과 소유자를 확인하고, 이미 회전한 Refresh 토큰이 다시 오면 탈취 가능성으로 보고 같은 계열 전체를 폐기하기로 했습니다.

행동 비밀번호를 Argon2id 로 저장하고, 목록·단건·집계·내보내기를 서버가 확인한 사용자 ID 로 제한했습니다. 기존 기록은 백업한 뒤 새 계정으로 옮겼고, 비밀번호를 바꾸면 기존 세션을 모두 폐기합니다.

결과 백엔드 테스트 319 개와 프론트엔드 76 개를 통과했고, 다른 계정 자료 접근과 재사용된 토큰이 거절되는 동작을 요청·응답 증거로 남겼습니다.

기술 React · TypeScript · Flask · PostgreSQL · Argon2id · Docker

[서비스 체험 · 첫 접속 느낌](#) · [인증 설명서](#) · [공개 저장소](#)

5 번 과제 · 대화 없이 넘긴 AI 인계

2026.08 - 2026.09

역할 실험 설계 · 프론트엔드 구현 · 검증 · 배포

담당 고정 검사 10 개 설계, 원본과 완성 카드를 겹쳐 보는 비교 슬라이더 구현, 인계 문서 7 개 항목 구성, 단위·E2E 회귀 검증, GitHub Pages 배포

능력 자기동기력

상황 AI 를 바꿀 때마다 사람이 같은 맥락을 다시 설명해야 했고, 대화를 넘기지 않고도 다음 AI 가 같은 프로젝트를 안전하게 이어갈 수 있는지는 확인된 적이 없었습니다.

판단 누가 더 빠르니 겨루는 대신 기능 범위와 고정 검사 10 개를 먼저 정하고, 대화를 넘기지 않은 채 코드·검사·인계 문서만으로 다음 AI 가 이어받는지 확인하기로 했습니다.

행동 실행하지 않은 검사는 통과로 적지 않는다는 기준을 먼저 세우고, 앞선 AI 가 검사 1~3 까지만 구현한 뒤 계획대로 멈추게 했습니다. 다음 AI 는 인계 문서만 읽고 검사 4~10 과 전체 회귀 검사를 마쳤습니다. 다음 AI 가 금지 규칙보다 앞선 AI 의 작업 기록을 먼저 읽은 절차 위반은 통과 결과로 덮지 않고 작업 기록과 공개 화면에 그대로 남겼습니다.

결과 고정 검사 10 개를 모두 통과하고 단위 테스트 85 개와 Chromium E2E 38 개를 유지했으며, 사람이 코드를 다시 고치거나 같은 내용을 재설명한 일은 없었습니다.

기술 React 19 · Vite 8 · Canvas API · Playwright

[사이트 체험](#) · [인계 결과 보기](#) · [공개 저장소](#)

수업 실습 · 경보와 권한 회수를 붙인 보안 게시판

2026.09

역할 보안 기능 설계·구현 · 자동화 연동 · 검증

담당 등급별 접근 제어와 관리자 안전장치, n8n 로그인 경보 봇과 메신저 3 종 알림, 수업에서 받은 과잉권한 자동 회수 코드의 이식, Graylog SYN Flood 탐지 연동

능력 자기조절력

상황 수업에서 받은 게시판에 등급별 접근 제어를 넣고, 과잉권한을 자동으로 회수하는 수업 코드를 옮겨 붙여야 했습니다. 자동 회수가 잘못 돌면 관리자까지 강등해 아무도 회원을 관리할 수 없게 될 수 있었습니다.

판단 권한을 자동으로 회수하는 기능일수록 관리자가 스스로 잠기는 사고부터 막아야 한다고 보고, 자기 강등·자기 삭제·마지막 관리자 강등을 금지한 규칙을 회수 봇에도 그대로 적용했습니다.

행동 회수 봇은 출력만 확인, Graylog 신고만, 실제 강등의 3 단계로 나눠 옵션을 줄 때만 권한을 빼앗게 했고, 증적 캡처와 경보 전송기가 API 키와 Webhook 주소를 화면과 오류 메시지에 남기지 않게 했습니다. SYN Flood 실습에서는 경보가 도착한 것만 보지 않고 실행 기록과 대조해, 공격이 허용으로 표시되고 실제 출발지와 개수가 빠지던 문제를 고쳤습니다.

결과 자동 테스트 45 개를 통과했고, Kali 에서 보낸 2 초 SYN Flood 35,033 개가 출발지·개수와 함께 Graylog, n8n, 메신저 3 종, 게시판 기록까지 이어지는 것을 확인했습니다.

기술 Flask · MySQL · JWT · n8n · Graylog · Docker

[공개 저장소](#)

4 번 과제 · GAME PULSE 데이터 보드

2026.08 - 2026.09

역할 데이터 수집 설계 · 프론트엔드 구현 · 검증 · 배포

담당 Steam 동시접속·매출·할인·방송 네 갈래 수집과 저장, React 19 화면 구현, 장애 재현과 복구 처리, GitHub Pages 배포

능력 자기조절력

상황 여러 곳에서 모은 숫자를 한 화면에 모으면, 그 값이 언제 어디서 왔는지와 수집이 실패했을 때 무엇이 보이는지가 불분명해집니다.

판단 Stack Overflow 대신 Steam 을 골라 동시접속·매출·할인·방송 네 축으로 좁히고, 수집이 실패하면 모르는 값을 지어내지 않고 마지막 정상값과 그 시각을 그대로 보여 주기로 했습니다.

행동 값마다 출처·측정 시각·데이터 상태·계산 근거를 함께 저장하고, timeout 을 포함한 장애 5 가지를 재현해 복구 흐름을 검사했습니다.

결과 회귀 테스트 317 개를 모두 통과하고 과제 조건 35 개의 감사 경로를 남겼습니다. 공개 화면에서 원자료·저장값·계산값·화면값을 손으로 대조할 수 있습니다.

기술 React 19 · Vite 8 · Node 22 · GitHub Actions · GitHub Pages

[사이트 체험](#) · [공개 저장소](#)

2 번 과제 · SOC SHIFT:30 관제 판단 게임

2026.08

역할 기획 · 프론트엔드 구현 · 테스트 · 배포

담당 30 초 판정 규칙 설계, React 18·TypeScript 구현, 키보드·마우스·터치 조작, 서버 없이 GitHub Pages 배포

능력 자기동기력

상황 보안 관제의 판단을 짧게 체험하게 하려면, 정답률 하나로만 점수를 매겨서는 실제로 무엇을 잘못했는지 드러나지 않습니다.

판단 틀리는 방식을 오탐과 미탐으로 나누어 따로 세면, 아무거나 막아서 버틴 판과 제대로 가려낸 판이 같은 점수가 되지 않는다고 판단했습니다.

행동 경보 한 건을 3 초 안에 통과·차단으로 정하는 30 초 근무를 만들고, 결과 화면에서 오탐과 미탐을 따로 세어 막대로 보여 주었습니다.

결과 서버 없이 동작하는 구조로 만들고 자동 테스트 291 개를 통과했으며, 설치나 로그인 없이 링크만으로 플레이할 수 있게 공개했습니다.

기술 React 18 · TypeScript · Vite 8

[게임 실행](#) · [공개 저장소](#)